



UNIVERSIDAD
DE GRANADA

TRABAJO FIN DE GRADO
INGENIERÍA INFORMÁTICA

Estudio e implementación paralela de métodos semiimplícitos multipaso

Resolución de Ecuaciones de Advección-Reacción-Difusión

Autor

José Manuel Navarro Cuartero

Director

José Miguel Mantas Ruiz



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN

Granada, Septiembre de 2026

**Estudio e implementación paralela de métodos
semiimplícitos multipaso:
Resolución de Ecuaciones de Advección-Reacción-Difusión**

José Manuel Navarro Cuartero

Palabras clave: palabra_clave1, palabra_clave2, palabra_clave3,

Resumen

Poner aquí el resumen.

Project Title: Project Subtitle

First name, Family name (student)

Keywords: Keyword1, Keyword2, Keyword3,

Abstract

Write here the abstract in English.

Yo, **José Manuel Navarro Cuartero**, alumno de la titulación **Grado en Ingeniería Informática** de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 75896777C, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: José Manuel Navarro Cuartero

Granada a X de Septiembre de 2026.

D. **José Miguel Mantas Ruiz**, Profesor del Área de XXXX del Departamento de Lenguajes y Sistemas Informáticos de la Universidad de Granada.

Informa:

Que el presente trabajo, titulado **Estudio e implementación paralela de métodos semiimplícitos multipaso, Resolución de Ecuaciones de Advección-Reacción-Difusión**, ha sido realizado bajo su supervisión por **José Manuel Navarro Cuartero**, y autorizo la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expide y firma el presente informe en Granada a X de Septiembre de 2026.

El director:

José Miguel Mantas Ruiz

Agradecimientos

Poner aquí agradecimientos...

Índice

Introducción	1
Conceptos previos	3
Métodos numéricos y EDOs	3
Tipos de métodos numéricos	4
Método de Euler	5
Métodos de Resolución	8
Runge-Kutta	8
Adams-Bashford	10
Adams-Bashford-Moulton	12
Problemas	13
Paralelismo	14
Secuencial	14
OpenMP	14
CUDA	14
Mediciones	14

Introducción

Los métodos numéricos son un conjunto de técnicas y algoritmos matemáticos diseñados para encontrar soluciones aproximadas a problemas que, en muchos casos, no pueden resolverse de manera exacta mediante el análisis o su resolución sería muy costosa. Se utilizan asiduamente en diversos campos de la ciencia, ingeniería, economía y muchas otras disciplinas para resolver ecuaciones diferenciales, integrales, sistemas de ecuaciones, aproximación e interpolación, integración numérica entre otros. Es en el primero de estos casos en el que nos centraremos. Si bien es cierto que existen diversas técnicas para la resolución de ecuaciones diferenciales ordinarias y de sistemas de ecuaciones diferenciales, éstas solo se pueden aplicar cuando dichas ecuaciones se ajustan a un patrón concreto.

En el mundo de los métodos numéricos hay infinitud de formas de resolver las EDOs, y tantas o más aplicaciones de las mismas a problemas del mundo real. Escoger el método correcto tiene que ver tanto con la exactitud de la solución, como con su coste, de potencia de computación y tiempo.

Es en este último punto en el que nos centraremos en los siguientes capítulos, ya que si bien para problemas pequeños, algo tan simple como el método de Euler explícito podría bastarnos, posteriormente plantearemos problemas mucho más complejos, que requerirán soluciones que estén a la altura. Por tanto, no se trata solamente de hallar una solución, sino de hacerlo rápida y eficientemente.

Para esto, partiremos de 4 problemas cuya implementación secuencial y soluciones esperadas ya conocemos de forma previa, y aplicaremos para su resolución 3 métodos numéricos distintos, que a su vez implementaremos con dos tecnologías para paralelizar distintas, OpenMP y CUDA, es decir, paralelismo a nivel de CPU y GPU.

Nuestro objetivo será implementar de forma fiel y exacta tanto los problemas como los métodos de resolución, para posteriormente poder comparar las ganancias en tiempo de ejecución al pasar de una tecnología a otra.

La idea detrás de todo esto es relativamente simple; un algoritmo que utilice 4 o 16 hebras será mejor que uno secuencial. Y uno que utilice 256 será mejor que todos ellos. Aunque la realidad no siempre va de la mano de nuestras ideas, buscaremos conseguir las mayores ganancias aprovechando al máximo todas las herramientas que ponen a nuestra disposición tanto CUDA

como OpenMP. Haremos por tanto una serie de mediciones y buscaremos probar nuestra hipótesis.

Conceptos previos

Métodos numéricos y EDOs

Una ecuación diferencial es aquella que relaciona una función ($f(x)$ ó y), sus variables (x) y sus derivadas ($f'(x)$, dy/dx ó y'). Para resolver una ecuación diferencial se debe encontrar una función que satisfaga dicha ecuación. Dentro de las ecuaciones diferenciales nos encontramos las ecuaciones diferenciales ordinarias, EDOs, que contienen derivadas respecto a una sola variable independiente. Más adelante nos centraremos en este tipo en particular. En oposición a éstas estarían las ecuaciones diferenciales parciales, en las que la función $f(x)$ depende de más variables independientes. [Molero et al., 2007]

El orden de una ecuación diferencial equivale al orden de la mayor derivada que aparece en la ecuación. Además, será lineal cuando la variable dependiente y todas sus derivadas son de primer grado (no puede haber $f(x)^2$), y los coeficientes de la variable dependiente y sus derivadas dependen de la variable independiente.

A modo de ejemplo simple podríamos considerar una ecuación diferencial ordinaria de primer orden como:

$$\frac{dy}{dt} = -2y, y(0) = 1$$

Donde además de la EDO, se nos está dando su valor inicial. En este caso en particular, sabemos de forma analítica que su solución exacta es $y(t) = e^{-2t}$, y podríamos por tanto calcular cualquier y_n que deseemos, pero si por cualquier razón no pudiéramos resolverla analíticamente deberíamos recurrir a los métodos numéricos. A través de ellos, y partiendo del valor y_n que se nos proporciona, calcularíamos de forma iterativa y_{n+1} hasta llegar al valor n que busquemos.

Podríamos utilizar, por ejemplo, el método de Euler, del que hablaremos más adelante. Antes de ello podemos entrar en algo más de detalle respecto a algunos otros conceptos que iremos usando.

Tipos de métodos numéricos

Podemos dividir a los métodos en tres grupos, explícitos, implícitos y semiimplícitos. La división se hará en función de cómo se consigue el siguiente paso, es decir, de cómo obtenemos y_{n+1} a partir de y_n ; de si aparecen valores futuros o no en la fórmula que lo calcula. Tendremos como expresión general la siguiente: [Molero et al., 2007]

$$y_{n+1} = y_n + \Delta t \cdot \Phi(x_n, y_n, y_{n+1}, \Delta t) \quad (1)$$

En la cual será la función incremento Φ la que dirima a cuál de los grupos pertenece el método con el que estemos trabajando.

Los métodos explícitos son aquellos en los que siguiente término se puede obtener directamente, sólo haciendo cálculos con datos de los que ya se dispone. Dicho de otra forma, son aquellos en los que y_{n+1} no aparece en Φ , sólo aparece en la parte izquierda de la ecuación. Entre ellos nos encontraremos el Euler explícito, Runge-Kutta, o Adams-Bashford, los tres los veremos posteriormente. Los métodos explícitos suelen ser más fáciles de implementar, ya que no requieren resolver ecuaciones, y suelen ser más rápidos. Por otro lado, tienden a ser menos estables, especialmente en problemas rígidos (*stiff*).

Al otro lado del espectro nos encontraríamos a los métodos implícitos, que son aquellos en los que y_{n+1} sí aparece en la parte de la derecha de la igualdad, lo cuál nos obligaría a resolver una ecuación que podrá ser, o no, lineal. Estos métodos son muy estables, y son más adecuados para resolver problemas rígidos (*stiff*). Como parte negativa tienen que requieren resolver ecuaciones en cada paso, y a veces de forma iterativa, lo que los acaba haciendo mucho más costosos computacionalmente. Entre ellos nos encontramos algunos métodos como el Euler implícito, Adams-Moulton, o los métodos BDF.

Como tercer grupo tendríamos los métodos semiimplícitos. Éstos representan una especie de "solución" a los diferentes problemas que traen consigo los métodos explícitos e implícitos. En este grupo, aunque existe una parte implícita, ésta es más simple, ya que o bien la ecuación a resolver es lineal, o bien se separa la parte rígida para tratarla implícitamente y el resto se trata de forma explícita. Esto es útil cuando el problema tiene términos rígidos (*stiff*) como la difusión, que deben tratarse implícitamente para que el método sea estable incluso con pasos de tiempo grandes, y términos suaves o de comportamiento hiperbólico como la advección, que se pueden tratar explícitamente para evitar resolver sistemas grandes o no lineales. Los métodos semiimplícitos representan un compromiso entre la estabilidad de los explícitos y la velocidad de los implícitos. Como hemos mencionado, podremos dividir a la EDO en dos partes:

$$y' = f(x, y) = f_E(x, y) + f_I(x, y) \quad (2)$$

En este ejemplo f_E es la parte explícita, suave (no *stiff*), y f_I es la parte que queremos tratar implícitamente. Algunos métodos semiimplícitos son los métodos IMEX (*implicit-explicit*), los métodos linealmente implícitos Runge-Kutta (*LIRK*), o el método Adams-Bashford-Moulton (AB-AM predictor-corrector). Es con este último con el que trabajaremos.

Otra categorización con la que trabajaremos serán los métodos de un solo paso, y los métodos multipaso. En los métodos de un solo paso, para obtener el siguiente valor con el que se tratará, el siguiente paso, sólo se utiliza información relativa al paso anterior. Es decir, como podemos ver en 1, para obtener y_{n+1} sólo se utilizaría y_n .

Sin embargo, los métodos multipaso usan varios valores anteriores de la solución para calcular el siguiente paso en el tiempo, en lugar de basarse solo en el valor actual. Es decir, que para obtener y_{n+1} se podrían tomar pasos intermedios que posteriormente se desechen (como en Runge-Kutta), o utilizar pasos más antiguos como y_{n-1} o y_{n-2} . Para problemas de gran tamaño, los métodos multipaso permiten alcanzar resultados con gran precisión y de una forma más eficiente que los métodos de un paso como el de Euler o el de Runge-Kutta.

Método de Euler

Como hemos comentado, el método de Euler es uno de los métodos numéricos más simples de los que disponemos para la resolución de EDOs. Se trata de un método explícito de un solo paso que se construye a partir de una sencilla idea geométrica: aproximar el valor de la solución en cada nodo por el valor que proporciona la recta tangente a la solución trazada desde el punto correspondiente al nodo anterior. [Gallardo, 2018]

Emplearíamos la siguiente fórmula:

$$y_{n+1} = y_n + \Delta t \cdot f(t_n, y_n) \quad (3)$$

Aplicando la misma al ejemplo de valor inicial que mencionamos antes, $\frac{dy}{dt} = -2y$, $y(0) = 1$, sabemos que y_n corresponde a la aproximación numérica de $y(t_n)$, y Δt representa la variación en el tiempo paso a paso que estemos considerando. Además, en nuestro caso, $f(t_n, y_n)$ sería $-2 \cdot y_n$, y se nos da el valor inicial $t_0 = 0.0$. Por tanto, tomando estos valores, con un $\Delta t = 0.1$, y $t_f = 1.0$, calcularemos los sucesivos valores para $t = 0.1, 0.2, \dots, 1.0$.

El primer paso sería por tanto, partiendo de los valores iniciales, calcular el $t = 0.1$, a saber, $y_1 = y_0 + \Delta t \cdot f(t_0, y_0) = 1 + 0.1 \cdot (-2 \cdot 1) = 0.8$. Si esto lo repitiéramos de forma sucesiva obtendríamos los siguientes resultados, los cuales además compararemos con el valor real $y(t) = e^{-2t}$ para obtener el

error absoluto ε_a y el error relativo ε_r . Definiremos al error absoluto como la diferencia entre el valor real y nuestra aproximación, y al error relativo como el cociente entre el error absoluto y el valor real, en porcentaje.

Paso n	t_n	$y_n(\text{Euler})$	$y_n(\text{Exacta})$	ε_a	ε_r
0	0.0	Valor Inicial = 1.0		-	-
1	0.1	0.8	0.81873	0.01873	2.29
2	0.2	0.64	0.67032	0.03032	4.52
3	0.3	0.512	0.54881	0.03681	6.71
4	0.4	0.4096	0.44933	0.03973	8.84
5	0.5	0.32768	0.36788	0.04020	10.93
6	0.6	0.26214	0.30119	0.03905	12.97
7	0.7	0.20972	0.24660	0.03688	14.96
8	0.8	0.16777	0.20190	0.03412	16.90
9	0.9	0.13422	0.16530	0.03108	18.80
10	1.0	0.10737	0.13534	0.02796	20.66

Tabla 1: Comparativa entre Euler y valor real

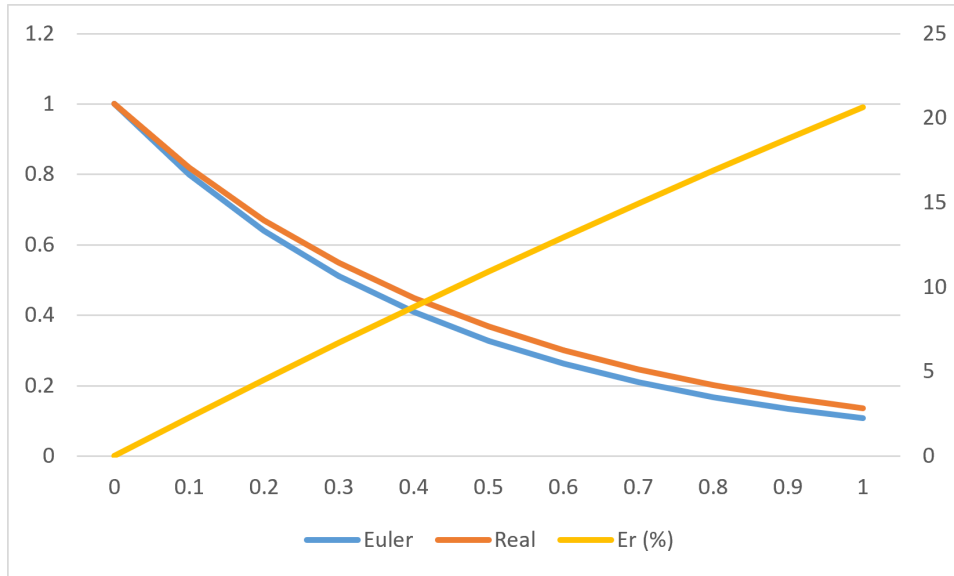


Figura 1: Comparativa entre Euler y valor real

Como podemos observar en la tabla 1 y en la figura 1, los valores que obtenemos con el Euler explícito son próximos a los reales (eje Y izquierdo); el error absoluto se mantiene estable por debajo de 0.05, e incluso disminuye. Sin embargo, el error relativo (eje Y derecho) aumenta de forma lineal con cada iteración, y para la última alcanza un preocupante 20 %. Esto nos indica

que para este problema, quizás el método numérico que hemos utilizado o el Δt que hemos considerado no son los más indicados.

Métodos de Resolución

Para resolver los diferentes problemas hemos implementado tres métodos distintos, a saber: Runge-Kutta, que es explícito y utiliza pasos intermedios ($K1, K2$) para obtener los nuevos pasos; Adams-Bashford, que también es explícito y multipaso, pero utiliza pasos antiguos (como f_{n-1} o f_{n-2}); y Adams-Bashford-Moulton, un predictor-corrector semiimplícito multipaso basado en Adams-Moulton (formalmente implícito) que utiliza Adams-Bashford para predecir y_{n+1} y sustituirlo en $f(t_{n+1}, y_{n+1})$.

Runge-Kutta

Ahora que hemos tratado el método de Euler, podremos dar el siguiente paso hacia Runge-Kutta. Aunque técnicamente ya lo hemos estado viendo, ya que los métodos de Runge-Kutta son generalizaciones de la fórmula básica de Euler. Por esta razón, se dice que el método de Euler es un método de Runge-Kutta de primer orden. [Segarra, 2020]

En cada uno de los órdenes de Runge-Kutta resolveremos una serie de pasos intermedios para hallar el valor final. El de orden 1, como hemos mencionado, coincidiría con Euler, como se puede ver en la ecuación 3.

Para ilustrar este punto, procedemos a pasar a la fórmula de segundo orden, en el que buscaremos encontrar constantes o parámetros w_1, w_2, α, β tal que la fórmula concuerda con un polinomio de Taylor de grado 2. [Zill, 2009]

$$\begin{aligned}y_{n+1} &= y_n + h \cdot (w_1 k_1 + w_2 k_2) \\k_1 &= f(x_n, y_n) \\k_2 &= f(x_n + \alpha h, y_n + \beta h k_1)\end{aligned}\tag{4}$$

Para el uso que haremos en este proyecto, basta decir que esto se puede hacer siempre que las constantes cumplan que:

$$\begin{aligned}w_1 + w_2 &= 1, \quad w_2 \alpha = \frac{1}{2} \text{ y } w_2 \beta = \frac{1}{2} \\w_1 &= 1 - w_2, \quad \alpha = \frac{2}{2w_2} \text{ y } \beta = \frac{1}{2w_2}\end{aligned}$$

Este es un sistema algebraico que posee tres ecuaciones y cuatro incógnitas, lo cual implica un número infinito de soluciones, donde $w_2 \neq 0$, y para nuestra aplicación práctica tomaremos $w_1 = w_2 = \frac{1}{2}$ y $\alpha = \beta = 1$, formando por tanto:

$$\begin{aligned} y_{n+1} &= y_n + h/2 \cdot (k_1 + k_2) \\ k_1 &= f(t_n, y_n) \\ k_2 &= f(t_n + h, y_n + k_1 \cdot h) \end{aligned} \tag{5}$$

De forma análoga, obtendremos el tercer orden:

$$\begin{aligned} y_{n+1} &= y_n + h/6 \cdot (k_1 + 4k_2 + k_3) \\ k_1 &= f(t_n, y_n) \\ k_2 &= f(t_n + h/2, y_n + k_1 \cdot h/2) \\ k_3 &= f(t_n + h, y_n - k_1 \cdot h + 2k_2 \cdot h) \end{aligned} \tag{6}$$

En el método de Runge-Kutta de orden 4 resolveremos diversos pasos que nos servirán para estimar de forma muy aproximada el resultado. Será en éste en el que nos centraremos a la hora de las mediciones. Partiendo de un valor inicial $dy/dt = f(t, y)$, $y(t_0) = y_0$, y de un valor de salto $h > 0$, calcularemos iterativamente los valores tal que:

$$\begin{aligned} y_{n+1} &= y_n + h/6 \cdot (k_1 + 2k_2 + 2k_3 + k_4) \\ k_1 &= f(t_n, y_n) \\ k_2 &= f(t_n + h/2, y_n + k_1 \cdot h/2) \\ k_3 &= f(t_n + h/2, y_n + k_2 \cdot h/2) \\ k_4 &= f(t_n + h, y_n + k_3 \cdot h) \end{aligned} \tag{7}$$

Aplicado en un contexto de programación, tendremos una clase *RungeKutta* con una función principal que aplicará el cálculo y algunas auxiliares que se explicarán más adelante. Dicha función principal tendrá el siguiente diseño, en el que se harán diversas llamadas a la evaluación de la derivada de cada problema en cuestión (aquí lo llamaremos *feval*):

Algoritmo 1 Runge-Kutta de Orden 4

```

 $y_n \leftarrow y_0$ 
 $t_n \leftarrow t_0$ 
while  $t_n < t_f$  do
     $K1 \leftarrow feval(t_n, Y_n)$ 
     $K2 \leftarrow feval(t_n + h/2, Y_n + K1 \cdot h/2)$ 
     $K3 \leftarrow feval(t_n + h/2, Y_n + K2 \cdot h/2)$ 
     $K4 \leftarrow feval(t_n + h, Y_n + K3 \cdot h)$ 
     $y_n \leftarrow y_n + h/6 \cdot (K1 + 2 \cdot K2 + 2 \cdot K3 + K4)$ 
     $t_n \leftarrow t_n + h$ 
end while
 $y_f \leftarrow y_n$ 

```

Como podemos ver, la aplicación del algoritmo es relativamente fácil de entender, se trabaja con 4 vectores auxiliares en los que hacemos los cálculos intermedios, que luego se suman ponderadamente. Como cada evaluación K depende de la anterior, no podremos hacer una operación vectorial en la que se evalúen los cuatro K en paralelo, sino que será en las otras operaciones, las evaluaciones de cada miembro, las sumas, y las multiplicaciones de cada vector, en las que podremos entrar a trabajar el paralelismo para mejorar la eficiencia de nuestro código.

Adams-Bashford

A diferencia de Runge-Kutta, donde sólo considerábamos un valor y_n para obtener y_{n+1} , en los métodos de Adams-Bashford, al ser multipaso, sí tendremos en cuenta más puntos, como y_{n-1} o y_{n-3} . En este proyecto nos hemos centrado en los métodos de paso fijo, es decir, que la diferencia entre y_n y y_{n-1} es constante (e igual a Δt). Si bien existen otros métodos de paso variable, no serán tratados.

De forma general, tenemos que un método lineal de k pasos viene determinado por una expresión tal que: [Molero et al., 2007]

$$\sum_{j=0}^k \alpha_j \cdot z_{n+j} = h \cdot \sum_{j=0}^k \beta_j \cdot f(x_{n+j}, z_{n+j}) , n \geq 0 \quad (8)$$

Para poder aplicar esta fórmula necesitaremos unos valores de arranque, que serán iguales al valor de k . Como en los problemas de valor inicial sólo se proporciona el primero de ellos y_0 , los valores restantes deberemos conseguirlos aplicando otro método (que no sea multipaso), como por ejemplo, Runge-Kutta.

La expresión general de un método de Adams-Bashford de k pasos es: [Molero et al., 2007]

$$z_{n+1} = z_n + h \cdot (\gamma_0 \cdot \nabla^0 + \gamma_1 \cdot \nabla^1 + \dots + \gamma_{k-1} \cdot \nabla^{k-1}) f_n$$

donde

$$\gamma_i = \frac{1}{i! h^{i+1}} \int_{x_n}^{x_{n+1}} (x - x_n) \dots (x - x_{n-i+1}) \cdot dx \quad (9)$$

Aplicando la anterior ecuación 9, obtendríamos los distintos órdenes con los que trabajaremos. Al igual que con Runge-Kutta, el Adams-Bashford de orden 1 coincide con Euler explícito (ecuación 3). De igual manera, obtendríamos Adams-Bashford de 2 pasos:

$$y_{n+1} = y_n + h/2 \cdot (3f_n - f_{n-1}) \quad (10)$$

Este será el primer caso en el que vemos aplicada la teoría de los métodos multipaso, ya que, para obtener el paso y_2 , por ejemplo, deberíamos tener de antemano f_1 y f_0 , para los que a su vez necesitaríamos (a través del arranque con Runge-Kutta) y_1 y y_0 . Una vez hecho el arranque inicial para obtener y_2 , ya sí podríamos operar iterativamente sólo con Adams-Bashford. El de orden 3 sería:

$$y_{n+1} = y_n + h/12 \cdot (23f_n - 16f_{n-1} + 5f_{n-2}) \quad (11)$$

Y por último tendríamos la expresión de orden 4, que será con la que principalmente trabajaremos:

$$y_{n+1} = y_n + h/24 \cdot (55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}) \quad (12)$$

Llevándonos este método a la programación, tendremos una clase *Adams-Bashford* con una función *aplicar* que se encargará de, primero, realizar un arranque con Runge-Kutta de orden 4 (algoritmo 1) y posteriormente poner en funcionamiento un bucle que ejecute la expresión tantas veces como esté establecido. En función del lenguaje utilizado y del tipo de paralelismo, las operaciones vectoriales funcionarán de una forma u otra, pero el planteamiento será el mismo, con llamadas a la evaluación de cada problema en cuestión (*feval*). El algoritmo para aplicar el método, siendo k el orden del mismo, será:

Algoritmo 2 Adams-Bashford general

```

 $y_n \leftarrow \text{arranqueRK4}(y_{n-1}) \quad \triangleright 1 \leq n < k$ 
 $f_n \leftarrow \text{feval}(y_n) \quad \triangleright 0 \leq n < k$ 
 $t_n \leftarrow t_0 + (k-1) \cdot h$ 
while  $t_n < t_f$  do
     $y_k \leftarrow \text{AdamsBashford}_k(y_{k-1}, f_n) \quad \triangleright \text{Aplicar orden correspondiente}$ 
     $f_k \leftarrow \text{feval}(y_k)$ 
     $\text{swap}(y_k, y_{k-1})$ 
     $\text{swap}(f_n, f_{n-1}) \quad \triangleright 0 \leq n < k$ 
     $t_n \leftarrow t_n + h$ 
end while
 $y_f \leftarrow y_{k-1}$ 

```

La idea detrás del algoritmo es que, tras el arranque de los $k-1$ vectores y_n , y la evaluación de los k vectores f_n , donde k es igual al orden de A-B que estemos aplicando, entrar en el bucle iterativo. En él, iteraremos sobre 2 vectores y_n , a saber, y_k y y_{k-1} , y sobre $k+1$ vectores f_n .

Una vez en el bucle, calculamos el nuevo valor de y_k , y con él, f_k . Posteriormente se hará un swap de vectores tal que y_k pasa a ser y_{k-1} , y todos los vectores f_n pasan a ser f_{n-1} . Así tendremos los datos preparados para la siguiente iteración.

Adams-Bashford-Moulton

Los métodos de Adams-Moulton tienen un funcionamiento similar a los de Adams-Bashford, con la salvedad de que estos son implícitos. Esto quiere decir que para calcular y_{n+1} se hace uso de $f(t_{n+1}, y_{n+1})$. Para resolver este problema utilizaremos un sistema Predictor-Corrector dándonos un método semiimplícito.

La expresión general de un método Adams-Moulton puro de k pasos es: [Molero et al., 2007]

$$z_{n+1} = z_n + h \cdot (\gamma_0 \cdot \nabla^0 + \gamma_1 \cdot \nabla^1 + \dots + \gamma_k \cdot \nabla^k) f_{n+1}$$

donde

$$\gamma_0 = 0 \text{ y } \gamma_i = \frac{1}{i!h^{i+1}} \int_{x_n}^{x_{n+1}} (x - x_{n+1}) \dots (x - x_{n-i+2}) \cdot dx \quad (13)$$

Y por tanto, las ecuaciones correspondientes a los mismos son las siguientes: [Segarra, 2020]

Adams-Moulton orden 0 (AM1)

$$y_n = y_{n-1} + h \cdot f_n \quad (14)$$

Adams-Moulton orden 1 (AM2)

$$y_{n+1} = y_n + h/2 \cdot (f_{n+1} + f_n) \quad (15)$$

Adams-Moulton orden 2 (AM3)

$$y_{n+1} = y_n + h/12 \cdot (5f_{n+1} + 8f_n - f_{n-1}) \quad (16)$$

Adams-Moulton orden 3 (AM4)

$$y_{n+1} = y_n + h/24 \cdot (9f_{n+1} + 19f_n - 5f_{n-1} + f_{n-2}) \quad (17)$$

Adams-Moulton orden 4 (AM5)

$$y_{n+1} = y_n + h/720 \cdot (251f_{n+1} + 646f_n - 264f_{n-1} + 106f_{n-2} + 19f_{n-3}) \quad (18)$$

Será en el AM5 en el que nos centraremos principalmente.

Es intuitivamente obvio que llegados a este punto no podríamos resolver estos métodos con las herramientas de las que disponemos. ¿Cómo llegamos a f_{n+1} sin disponer todavía de y_{n+1} ? Para resolver este problema, empleamos un sistema Predictor-Corrector mediante el cual utilizamos Adams-Bashford para hacer una primera aproximación de f_{n+1} , y la evaluamos para posteriormente hacer una segunda aproximación usando Adams-Moulton. El esquema será el siguiente, donde k es igual al orden:

Algoritmo 3 Adams-Bashford-Moulton general

```
 $y_n \leftarrow \text{arranqueRK4}(y_{n-1}) \quad \triangleright 1 \leq n < k$   
 $f_n \leftarrow \text{feval}(y_n) \quad \triangleright 0 \leq n < k$   
 $t_n \leftarrow t_0 + (k-1) \cdot h$   
while  $t_n < t_f$  do  
   $y_{pred} \leftarrow \text{AdamsBashford}_k(y_{k-1}, f_n) \quad \triangleright \text{Aplicar orden } k$   
   $f_{pred} \leftarrow \text{feval}(y_{pred})$   
   $y_{corr} \leftarrow \text{AdamsMoulton}_k(y_{k-1}, f_{pred}) \quad \triangleright \text{Aplicar orden } k$   
   $f_{corr} \leftarrow \text{feval}(y_{corr})$   
   $y_k \leftarrow \text{AdamsMoulton}_k(y_{k-1}, f_{corr}) \quad \triangleright \text{Aplicar orden } k$   
   $f_k \leftarrow \text{feval}(y_k)$   
   $\text{swap}(y_k, y_{k-1})$   
   $\text{swap}(f_n, f_{n-1}) \quad \triangleright 0 \leq n < k$   
   $t_n \leftarrow t_n + h$   
end while  
 $y_f \leftarrow y_{k-1}$ 
```

El funcionamiento del algoritmo acaba siendo muy similar al esquema de Adams-Bashford, en tanto que hacemos el arranque con Runge-Kutta y posteriormente iteramos sobre los vectores y_n y f_n . En este caso, añadimos además los vectores predictor y corrector, que nos permiten aplicar la idea central de este método, manteniendo una buena eficiencia pero con resultados más precisos.

Problemas

Para poner en funcionamiento la implementación de estos métodos y de la paralelización de la que hablaremos a continuación, hemos tomado cuatro problemas distintos que nos permitan probar extensivamente ambas tecnologías, tanto OpenMP como CUDA. Los problemas en cuestión son:

SimpleAdvDiff

AdvDiff1D

Brusselator1D, el modelo brusselator es un tipo de modelo de reacción-difusión que fue desarrollado por la Brussels School. Este sistema está diseñado para analizar el comportamiento de sistemas químicos que presentan oscilación no lineal. [Hunsdorfer and Verwer, 2003]

Brusselator2D, este modelo es similar a su versión unidimensional, pero incorpora una componente extra reactiva. Además, está discretizado uniformemente en una malla de $N \times N$ puntos.

Paralelismo

Secuencial

OpenMP

CUDA

Mediciones

Bibliografía

- [Gallardo, 2018] Gallardo, J. M. (2018). *Análisis Numérico de Ecuaciones Diferenciales*. Universidad de Málaga.
- [Hunsdorfer and Verwer, 2003] Hunsdorfer, W. and Verwer, J. G. (2003). *Numerical Solution of Time-Dependent Advection-Diffusion-Reaction Equations*, volume 33. Springer-Verlag.
- [Molero et al., 2007] Molero, M., Alcaide, A., Menarguez, T., and Luis, S. (2007). *Análisis Matemático para Ingeniería*. Pearson Educación.
- [Segarra, 2020] Segarra, J. (2020). Runge-kutta y adams-bashford en mathematica. *Revista Ingeniería, Matemáticas y Ciencias de la Información*, 7:13–32.
- [Zill, 2009] Zill, D. G. (2009). *Ecuaciones diferenciales con aplicaciones de modelado*. Brooks/Cole, Cengage Learning.

Glosario de Tablas

1.	Comparativa entre Euler y valor real	6
----	--	---

Glosario de Imágenes

1.	Comparativa entre Euler y valor real	6
----	--	---